웹 개발자 부트캠프 과정

CodingOn

REST API

복습. API 란?

- Application Programming Interface
- 한 프로그램에서 다른 프로그램으로 정보를 주고받기 위한 규약(약속)
- 모든 프로그램은 API를 가질 수 있음
  - Windows API: 윈도우 운영체제 기능 사용 가능 (ex. 알림, 설치, 삭제 등)
  - Database 관리 프로그램 API: DB 입출력 기능 사용 가능
  - XX 프로그램 API: XX를 사용 가능

복습. REST

- 웹 서비스 개발에 널리 사용되는 아키텍처 스타일
- HTTP 프로토콜 위에서 동작
- HTTP URI를 통해 자원(Resource) 명시, HTTP Method를 통해 자원에 대한 CRUD 작업을 수행
- 자원을 이름(자원의 표현)으로 구분하여 해당 자원의 상태(정보)를 주고 받는 모든 것을 의미

resource(자원)? method?

resource : 내가 필요한 데이터, 모든 데이터들은 특정 장소에 보관되어 있음 (HTTP URI를 통해 자원을 명시)

method : CRUD 중 어떤 방식으로 요청할 건지

(참고) CRUD

	Create	Read	Update	Delete
블로그 게시글	생성	읽기	수정	삭제

복습. REST API

- representational state transfer API
- REST 아키텍처 스타일을 따르는 API
  - 즉, 클라이언트와 서버 간의 통신 방식을 정의할 때 REST 원칙을 따름
- **REST 기반으로 서비스 API를 구현**한 것
  - REST 를 적용한 웹 서비스를 **RESTful 웹 서비스**라고 함

GET	/movies	Get list of movies
GET	/movies/:id	Find a movie by its ID
POST	/movies	Create a new movie
PUT	/movies	Update an existing movie
DELETE	/movies	Delete an existing movie

REST 규칙 (1)

- URL로 리소스 식별
  - 투두 리스트 목록을 식별하는 URL
  - ex. http://codingon.example.com/todos

  - 2번 투두 리스트를 식별하는 URL
  - ex. http://codingon.example.com/todos/t02

  - 클래스 04 강의실의 투두 리스트를 식별하는 URL
  - ex. http://codingon.co.kr/class/c04/todos/t02

REST 규칙 (2)

- HTTP 메서드로 리소스 조작
  - **GET** : 조회
  - **POST** : 생성
  - **PUT**, **PATCH** : 수정
  - **DELETE** : 삭제

- Quiz. *POST http://codingon.example.com/todos/t100* 요청의 의미는?
  - T100 투두를 생성하는 요청

REST 규칙 (3)

- **Request** 의 데이터 형식

```
POST /api/users HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Content-Type: application/json
Content-Length: 57
Accept: application/json

{
  "name": "Jane Doe",
  "email": "jane.doe@example.com"
}
```

요청 라인

요청 헤더

요청 바디

- **Response** 의 데이터 형식

```
HTTP/1.1 201 Created
Date: Mon, 08 Jul 2024 12:30:15 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Type: application/json
Content-Length: 121
Location: /api/users/1001
Access-Control-Allow-Origin: *

{
  "id": 1001,
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "created_at": "2024-07-
08T12:30:15Z"
}
```

상태 라인

응답 헤더

응답 바디

REST 규칙 (3)

- **Request** 의 데이터 형식

```
POST /api/users HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Content-Type: application/json
Content-Length: 57
Accept: application/json

{
  "name": "Jane Doe",
  "email": "jane.doe@example.com"
}
```

요청 라인 : 요청의 개요 (HTTP 메소드, URL 경로, HTTP 버전)

요청 헤더 : 요청에 대한 부가적인 정보

Host: 요청을 보내는 서버의 도메인 이름
User-Agent: 클라이언트 애플리케이션 정보
Content-Type: 본문 데이터의 형식 (여기서는 JSON)
Content-Length: 본문의 길이 (바이트 단위)
Accept: 클라이언트가 받아들일 수 있는 응답 형식

요청 바디

: 서버가 요청을 처리할 때 필요한 데이터

*빈 줄: 헤더와 본문 구분

REST 규칙 (3)

- **Response** 의 데이터 형식

```
HTTP/1.1 201 Created
Date: Mon, 08 Jul 2024 12:30:15 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Type: application/json
Content-Length: 121
Location: /api/users/1001
Access-Control-Allow-Origin: *

{
  "id": 1001,
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "created_at": "2024-07-
08T12:30:15Z"
}
```

*빈 줄: 헤더와 본문 구분

상태 라인 : 응답의 결과 (HTTP 버전, 상태 코드, 상태 메세지)

응답 헤더 : 응답의 부가적인 정보

Date: 응답이 생성된 날짜와 시간
Server: 웹 서버 소프트웨어 정보
Content-Type: 응답 본문의 미디어 타입 (JSON)
Content-Length: 응답 본문의 길이 (바이트)
Location: 새로 생성된 리소스의 URI
Access-Control-Allow-Origin: CORS 정책 (모든 도메인에서 접근 허용)

응답 바디
: 클라이언트가 요청한 데이터

REST 규칙 (4)

- **상태 코드(Status code)** 활용하기
- 상태코드 란?
  - 3자리의 숫자로 클라이언트의 요청에 대한 서버의 처리 결과

코드	의미
1xx	요청이 수신되어 처리중임
2xx	요청이 성공적으로 처리되어 종료
3xx	다른 처리의 호출이 필요
4xx	요청에 오류가 있음 (클라이언트 측 에러)
5xx	서버가 유효한 요청을 처리하지 못함 (서버 측 오류)

보충. 주요 상태 코드

상태코드 / 상태 메시지	의미
200 OK	요청이 성공적으로 처리되었음
201 Created	새로운 리소스가 성공적으로 생성되었음 (주로 POST 요청에서 사용)
204 No Content	요청이 성공적으로 처리되었지만, 응답 본문에 반환할 내용이 없음 (DELETE 요청 후 사용 가능)
400 Bad Request	클라이언트의 요청이 잘못되었음 (ex. 잘못된 요청 구문 등)
401 Unauthorized	인증이 필요한 리소스에 대해 인증되지 않은 요청
403 Forbidden	서버가 요청을 이해했지만 수행을 거부했음 (ex. 권한 부족으로 인한 접근 거부)
404 Not Found	요청한 리소스를 서버에서 찾을 수 없음
405 Method Not Allowed	요청한 HTTP 메소드가 해당 리소스에 대해 허용되지 않음
409 Conflict	요청이 현재 서버의 상태와 충돌함 (ex. 다른 클라이언트가 동시에 갱신하는 경우)
500 Internal Server Error	서버에 예기치 않은 상황이 발생하여 요청을 처리할 수 없음
503 Service unavailable	서버가 일시적으로 요청을 처리할 수 없음 (ex. 과부하, 점검 등)

REST 규칙 (5)

- 헤더 활용하기
  - Request 와 Response 에서 Header 를 이용해 부가 정보를 기입 가능

- Request Header
  - Accept: 응답으로 받고 싶은 데이터 형식 지정
  - Authorization: 인증하기 위한 정보(ID나 비밀번호 등)를 지정
  - Content-Type: 요청의 바디 데이터 형식 지정

- Response Header
  - Content-Type: 응답 바디의 데이터 형식 지정
  - Location: 새로 등록한 리소스의 URL 지정

REST 규칙 (6)

- **서버 측을 무상태(stateless)** 로 만들기
  - 서버는 클라이언트의 요청을 처리한 후 상태를 보관하지 않음
  - 즉, 클라이언트가 서버에 요청을 보낼 때 이전 요청의 영향을 받지 않음
  - 요청할 때 마다 서로 다른 요청으로 인식되어 이전 작업 내용을 다음 요청 시 사용 불가

  - ex. 로그인 요청 후, 다음 요청에서 로그인 정보를 서버에게 다시 알려줘야 함
  - 왜냐하면 서버는 로그인 상태를 유지하지 못하므로!!!

  - 서버가 상태를 유지해야 한다면 데이터베이스에 리소스를 저장해야 함
  - 서버는 여러 클라이언트가 요청을 하므로 이전 요청의 상태를 유지할 수 없음

DTO

DTO (Data Transfer Object)

- 계층 간 데이터 교환을 위해 사용하는 객체
  - 즉, 데이터를 옮기기 위해 사용하는 전달자 역할
  - 데이터 전달 목적으로만 사용되므로 다른 로직을 가지지 않는 순수한 데이터 객체
- 일반적으로 getter/setter 메서드로만 구성

- 사용 예시
  - 데이터베이스 조회 결과를 담아 전달하는 용도로 자주 사용
  - 네트워크 통신 시 데이터 직렬화에 사용 가능

```
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class DTOExample {
    private String title;
    private String content;
    private String writer;
}
```

DTO 예제 코드

```java
public class DTOExample {
    private String title;
    private String content;
    private String writer;

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getContent() {
        return content;
    }

    public void setContent(String content) {
        this.content = content;
    }

    public String getWriter() {
        return writer;
    }

    public void setWriter(String writer) {
        this.writer = writer;
    }
}
```

```java
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class DTOExample {
    private String title;
    private String content;
    private String writer;
}
```

클래스 선언 위에 @Getter, @Setter 어노테이션 추가

- @Getter: 모든 필드에 대한 getter 메소드 자동 생성
- @Setter: 모든 필드에 대한 setter 메소드 자동 생성

Lombok을 이용해 **간결한 코드 작성**이 가능하며,
보일러플레이트 코드를 줄이고, **클래스의 본질 구조에 집중** 가능!

VO

VO (Value Object)

- DTO와 유사하지만, VO는 read-Only 속성을 갖고 있는 객체
- 값 자체를 표현하는 불변(immutable) 객체
  - 한 번 생성 되면 내부 값을 변경할 수 없음 -> getter 메서드만 가짐
  - 따라서, 객체의 속성 값이 같으면 같은 객체로 간주
  - equals() 와 hashCode() 메서드를 오버라이드하여 값 비교 수행
- 일부 비즈니스 로직 포함 가능

- 사용 예시
  - 날짜 범위: 시작 날짜와 종료 날짜를 하나의 객체로 표현
  - 주소: 국가, 도시, 상세 주소 등을 하나의 객체로 표현

VO 예제 코드

```java
import java.util.Objects;

/**
 * 2D 좌표를 표현하는 간단한 VO (Value Object) 클래스
 */
public final class Point {
    private final int x;
    private final int y;

    /**
     * Point 객체를 생성합니다.
     * @param x x 좌표
          * @param y y 좌표
          */
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // Getter 메서드들
        public int getX() {
        return x;
    }

    public int getY() {
        return y;
    }
```

```java
    /**
     * 현재 점에서 다른 점까지의 거리를 계산(비즈니스 로직)
     * @param other 다른 Point 객체
          * @return 두 점 사이의 거리
          */
    public double distanceTo(Point other) {
        int dx = this.x - other.x;
        int dy = this.y - other.y;
        return Math.sqrt(dx*dx + dy*dy);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Point point = (Point) o;
        return x == point.x && y == point.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }

    @Override
    public String toString() {
        return "Point(" + x + ", " + y + ")";
    }
}
```

DTO vs. VO

	DTO (Data Transfer Object)	VO (Value Object)
목적	계층간 데이터 전송	도메인 내 값 표현 불변의 복합 값 표현
동등성	일반적으로 참조 동등성 속성값이 같아도 다른 객체로 취급 가능	값 동등성 속성 값이 같으면 같은 객체 취급
가변성	일반적으로 가변(mutable) setter 메서드를 통한 속성 변경 가능	불변(immutable) 생성 후 내부 상태 변경 불가
로직	비즈니스 로직을 포함하지 않음 주로 getter/setter 만 포함	간단한 도메인 로직 포함 가능 값과 관련된 연산이나 검증 로직 포함 가능